DMLScope: Uncovering Logical Bugs in Database DML via Approximate State-Relation–Driven Metamorphic Testing

Abstract

...

1 Introduction

Database Management Systems (DBMSs) are a fundamental component of modern data-driven applications, ranging from e-commerce and online banking to large-scale enterprise systems [8, 37]. They provide standardized interfaces, such as SQL, to support both data retrieval and data modification [5]. Given their critical role, bugs in DBMSs can lead to severe consequences, including incorrect query results, data corruption, and even security violations [13, 36]. Among various types of bugs, logical bugs are particularly critical because they do not crash the system, but silently lead to incorrect query results or incorrect database states, often remaining unnoticed for long periods [16, 26, 30]. Recent studies have demonstrated that even well-tested DBMSs still contain numerous previously unknown logical bugs, highlighting the difficulty of ensuring their correctness [7, 11, 15, 26–28, 30–32].

While existing research has made significant progress in detecting logical bugs in read-only queries [7, 11, 15, 26–28, 30–32], considerably less attention has been paid to state-modifying SQL statements. In this work, we focus on data manipulation language (DML) statements that directly change persistent database contents, including common operations such as `INSERT`, `UPDATE`, and `DELETE`, as well as DBMS-specific extensions such as `REPLACE` when supported. Unlike read-only queries that only return result sets, these statements produce persistent state changes, making their correctness harder to validate and their consequences more severe. For example, an incorrect `INSERT` may introduce spurious tuples, an erroneous `UPDATE` may propagate wrong values to a large number of records, and an unintended `DELETE` may remove data that should have been preserved, leading to irreversible inconsistencies or data loss. Testing such statements is challenging because their semantics are often intertwined with complex predicates, nested subqueries, joins, constraint enforcement, and integrity-related side effects such as cascades. As a result, logical bugs in DBMSs may silently affect which tuples are inserted, modified, or deleted, thereby producing incorrect database states without any explicit failure signal [30].

Logical bugs in DBMSs, especially those in DML statements, are difficult to detect automatically. A fundamental challenge lies in the *test oracle problem* [12], i.e., determining whether the execution of a given SQL statement produces a correct result or database state. While this problem has been extensively studied for read-only queries, it becomes significantly more challenging for DML statements, as correctness must be defined over state transitions rather than query outputs. To address this challenge, recent work DQE [30] proposes a differential testing approach that detects logical bugs by comparing the rows accessed by `SELECT`, `UPDATE`, and `DELETE` statements sharing the same predicate. The key insight of DQE is that these statements should access the same set of tuples under identical conditions, and any inconsistency indicates a potential logical bug. Although effective, DQE relies on cross-statement consistency and treats DML correctness indirectly through row-access equivalence. This design introduces several limitations: (1) it requires restricting SQL features to those commonly supported across different statement types, limiting its applicability to complex queries; (2) it cannot naturally capture value-level correctness (e.g., incorrect assignments in `UPDATE`); and (3) it may fail to detect bugs when multiple statements share the same erroneous execution path [11]. These limitations suggest that a more direct oracle over database state transitions is needed for DML testing.

There have also been several studies on detecting logical bugs in DBMSs [7, 11, 15, 26–28, 30–32]. For instance, TLP [27] partitions a query based on three-valued logic and checks equivalence between decomposed queries, while NoREC [26] rewrites queries into non-optimized forms to expose inconsistencies. These techniques are effective at detecting logical bugs in *read-only queries*, where correctness can be defined over result sets. However, their primary focus is on query equivalence or result-set relations, rather than the correctness of *state-modifying semantics*. In contrast, our work focuses on logical bugs in DML statements, where errors manifest as incorrect database state transitions, including wrong tuple selection, unintended modifications, and incorrect value assignments. Such bugs occur during the execution of state-modifying operations and cannot be effectively captured by existing query-centric testing approaches.

Inspired by Pinolo [11], which demonstrates that approximate semantic relations can serve as effective invariants for logical bug detection, we revisit this principle in the context of state-changing SQL. We term this paradigm ***A**pproximate **S**tate-**R**elation–Driven **M**etamorphic **T**esting* (ASRMT): for a statement and its over-/under-approximate variant, correctness is validated by whether their post-execution effects satisfy directional partial-order constraints over database state changes (e.g., subset/superset), rather than output equivalence. Any violation of these approximate state-relation invariants indicates a potential logical bug.

2 Background and Motivation

2.1 Background

Relational Database Management Systems. Relational Database Management Systems (DBMSs) are used to create, manage, and interact with relational databases [34]. In a relational DBMS, data is organized into structured tables (relations) with predefined columns and rows, enabling efficient storage, retrieval, and manipulation. Structured Query Language (SQL) is the standard language for interacting with relational DBMSs, supporting data definition, manipulation, and querying [33].

Structured Query Language. Relational DBMSs provide Structured Query Language (SQL) [33] as a unified interface for interacting with stored data, enabling a wide range of operations such as schema definition, data modification, and data retrieval. Based

on their functionality, SQL statements are commonly categorized into three main types.

Data Definition Language (DDL) statements are responsible for defining and modifying database schema objects. Typical examples include `CREATE TABLE`, `ALTER TABLE`, and `DROP TABLE`, which are used to create new relations, adjust existing structures, and remove schema objects, respectively.

Data Manipulation Language (DML) statements are designed for modifying the data stored in database tables. Representative operations include `INSERT` for adding new tuples, `UPDATE` for modifying existing records, and `DELETE` for removing tuples from a table.

Data Query Language (DQL) statements are used for retrieving data from the database. The most representative DQL statement is `SELECT`, which supports a rich set of query constructs, including filtering conditions (e.g., `WHERE`), ordering (`ORDER BY`), aggregation functions, join operations, and grouping (`GROUP BY`), enabling flexible and expressive data analysis over one or multiple tables.

Although these statement types are all expressed in SQL, they differ significantly in terms of supported operators and semantics. In particular, `SELECT` statements provide a much richer set of relational operators and compositional capabilities, while `UPDATE` and `DELETE` focus on state modification and typically support only a subset of these features. For instance, `UPDATE` statements in many DBMSs allow value assignments based on expressions involving joins or external tables, whereas `DELETE` statements are primarily restricted to specifying which rows to remove and do not support value-level transformations. Consequently, `UPDATE` can express complex data-dependent transformations, while `DELETE` is limited to tuple elimination without value propagation.

DML Bugs. DML statements directly modify persistent database state, and bugs in their execution may lead to incorrect state transitions, such as updating unintended tuples, assigning incorrect values, or deleting valid data. Compared with read-only query bugs, such bugs are particularly severe because they may cause irreversible data corruption or loss.

From a testing perspective, DML bugs can be broadly divided into crash bugs and logical bugs. Crash bugs manifest as explicit failures (e.g., crashes or runtime errors) and can be detected using failure-based oracles. In contrast, logical bugs do not trigger visible failures; instead, the DBMS completes execution successfully while producing an incorrect database state. These bugs are especially challenging to detect, as they require reasoning about the correctness of state changes rather than returned results, and thus suffer from a more difficult test oracle problem. In this work, we focus on logical bugs in DML statements, as they are both practically harmful and insufficiently explored by existing DBMS testing techniques.

2.2 Limitations of Existing Works

Existing research on detecting logical bugs mainly relies on automated testing techniques. However, designing an effective testing framework is challenging due to the *test oracle problem*—it is difficult to determine the correct result of a complex SQL query for comparison. To address this issue, researchers have explored two main categories of approaches [15, 19, 30–32]. While this problem has been extensively studied for read-only queries, it becomes significantly more difficult for DML statements, whose correctness must be defined over state transitions rather than query outputs.

Differential Testing. Classical differential testing executes the same query across multiple DBMSs and compares their outputs to detect inconsistencies [4, 29]. However, dialect differences, heterogeneous semantics, and system-specific behaviors often make cross-DBMS comparison difficult in practice [11]. Different from this cross-system setting, DQE [30] brings the differential-testing idea into a single DBMS by comparing multiple related statements within the same system to detect DML bugs. Specifically, it constructs `SELECT`, `UPDATE`, and `DELETE` statements sharing the same predicate, and checks whether they access the same set of tuples. However, it has several limitations. First, because its oracle requires row-access comparability across statement types, test generation must stay within the subset of SQL features that can be meaningfully aligned across `SELECT`, `UPDATE`, and `DELETE`, which limits coverage for complex queries. Second, row-access consistency only validates *which tuples* are touched, but not *how* the database state is changed; thus it cannot naturally capture value-level correctness, such as whether an `UPDATE` computes and propagates the correct new values. Third, since DQE detects disagreement rather than correctness itself, it may miss bugs when multiple related statements follow the same faulty execution path and therefore remain consistently wrong.

Metamorphic Testing. Another dominant line of work is metamorphic testing (MT), which constructs follow-up queries from a seed query and checks whether their outputs satisfy predefined semantic relations [7, 15, 26–28, 31, 32]. Representative approaches such as NoREC [26] and TLP [27] define metamorphic relations based on strict equivalence of query results. Although effective in bypassing the need for ground truth, such equivalence-based strategies constrain the search space of test cases and primarily validate surface-level correctness [11]. To overcome this limitation, Pinolo [11] extends metamorphic testing by introducing approximate relations, generating over- and under-approximate queries and checking whether their results satisfy subset/superset relationships. This relaxation significantly broadens the detectable bug space and inspires more flexible oracle design. Our work is inspired by this key insight. However, existing metamorphic testing techniques, including Pinolo, remain fundamentally query-result-centric: their correctness criteria are defined over returned result sets rather than over the state transitions induced by DML execution. Therefore, directly bringing approximate relations into DML testing is non-trivial. It requires not only generating semantically valid and directionally sound approximate DML statements, but also designing a new oracle that can compare their state effects rather than conventional query outputs. We elaborate on these challenges next.

2.3 Challenges

Challenge #1: Generating Complex and Valid DML Statements. To effectively expose deep logical bugs, testing must explore *complex* DML statements that exercise rich execution paths, such as joins, nested subqueries, and data-dependent value transformations. However, prior work such as DQE [30] mainly focuses on relatively

simple DML patterns, limiting its ability to cover complex semantics and deeper execution behaviors. Compared with read-only queries, generating valid and meaningful DML statements is substantially more challenging. A DML statement must not only be syntactically correct, but also satisfy various semantic constraints, including key consistency, type compatibility, source–target alignment, and integrity constraints imposed by the schema and the current database pre-state. Otherwise, the statement may fail at execution time or produce undefined state effects.

Challenge #2: Lack of State-Aware Test Oracles for DML. Unlike read-only queries, whose correctness can be defined over explicit result sets, DML statements do not produce a single observable output. Instead, their semantics are reflected in state transitions over the underlying database. This makes traditional output-based oracles inapplicable. A key difficulty is that the effect of a DML statement is not only implicit, but also multi-dimensional. A DML execution simultaneously determines which rows are selected as targets, which rows are actually modified, and how their values are changed. These aspects do not necessarily coincide and may even become ambiguous in the presence of join multiplicity or DBMS-specific execution policies. Therefore, unlike query testing, DML testing often lacks a canonical notion of expected output or a uniquely defined post-state to compare against. ✎ [lin: add a motivating example, matched-row] Existing work partially addresses this problem from an indirect perspective. DQE [30] uses row-access consistency to verify correctness, which can capture which tuples are affected but cannot naturally validate how the database state is changed. Therefore, the challenge is to design a state-aware test oracle that can capture relevant aspects of state changes and support meaningful comparison for DML statements.

2.4 Solutions

Solution to Challenge #1: Structured-Guided and Constraint-Aware DML Generation. To address the difficulty of generating complex and valid DML statements, we adopt a structure-first, constraint-guided generation strategy. We explicitly separates structural complexity from semantic validity. Specifically, we first construct structurally rich DML skeletons using a grammar that captures diverse SQL constructs, such as joins and nested subqueries, ensuring sufficient exploration of complex execution paths. We then progressively instantiate these skeletons under schema constraints and the current database pre-state, so that validity requirements—such as key consistency, type compatibility, and source–target alignment—are satisfied by construction. This design enables the systematic generation of DML statements that are both sufficiently complex and reliably executable for subsequent testing.

Solution to Challenge #2: State-Relation-Driven Metamorphic Testing. To overcome the lack of state-aware test oracles for DML, we propose ***A**pproximate **S**tate-**R**elation–Driven **M**etamorphic **T**esting* (ASRMT). Inspired by Pinolo [11], ASRMT extends approximate metamorphic testing from query outputs to database state transitions. The key idea is to avoid requiring a canonical exact post-state as the test oracle. Instead, ASRMT abstracts DML execution into state effects, i.e., observable and comparable representations of how a statement changes the database state. This abstraction enables us to focus on the correctness-relevant aspects of state transitions without directly comparing full final states. Based on this abstraction, we define approximate state relations as over- and under-approximation constraints over DML effects, and use them as metamorphic oracles. Finally, we execute original and mutated statements from the same pre-state and check whether their observed effects satisfy the expected approximation relations. In this way, ASRMT turns the DML oracle problem from exact post-state validation into relation checking over comparable state effects. The full model and its concrete instantiations are presented in Section 3.

2.5 Motivating Example

3 Approximate State-Relation–Driven Metamorphic Testing

To address the test oracle problem for state-modifying SQL statements, we propose ***A**pproximate **S**tate-**R**elation–Driven **M**etamorphic **T**esting* (**ASRMT**), a metamorphic testing paradigm that generalizes prior approximation-based techniques Pinolo [11] from query results to database state transitions. Unlike existing approaches that reason about equivalence or inclusion relations over result sets, ASRMT defines correctness over the effects of DML statements on database states. This shift is essential for testing data manipulation language (DML) statements, whose semantics are reflected by persistent state changes rather than returned tuples.

3.1 State Effect

Let S denote a database state, and let q be a DML statement. The execution of q over S produces a new state:

$$[\![q]\!](S) \to S'.$$

To characterize the observable behavior of a DML statement, we introduce the notion of *state effect*.

Definition 3.1 (State Effect)**.** Given a DML statement q and a database state S, the state effect of q over S, denoted by $E(q, S)$, is an abstract set-valued representation of the observable effect induced by executing q on S.

The notion of state effect is intentionally abstract. In our paper, it is instantiated in two forms, namely the matched-row effect and the changed-row effect, depending on which aspect of the DML semantics is to be observed.

Definition 3.2 (Matched-Row Effect)**.** Given a DML statement q and a database state S, the matched-row effect of q, denoted by $E_m(q, S)$, is the set of target-row tokens corresponding to the primary keys of rows structurally matched by q on S.

Example 3.1. Consider two tables in database state S:

$t_1(c_1, c_2) =$

c_1	c_2
1	5
2	8
3	9

$t_2(c_1, c_3) =$

c_1	c_3
1	5
2	20
3	15

Consider the DML statement:

q : **UPDATE** t_1 **JOIN** t_2 **ON** $t_1.c_1 = t_2.c_1$ **SET** $t_1.c_2 = t_2.c_3$
WHERE $t_2.c_3 > 10$

Let $PK(t_1) = \{c_1\}$ be the primary key of t_1. The set of target rows in t_1 structurally matched by q is:

$$E_m(q, S) = \{t_1{:}2,\ t_1{:}3\}.$$

Intuitively, the join and predicate select the rows in t_1 whose corresponding $t_2.c_3 > 10$, thus determining the intended target set independent of actual value updates.

The matched-row effect captures the *intended target set* of a DML statement. It is suitable when the mutation primarily changes row-selection semantics, such as predicate strengthening/weakening, join-condition mutation, or other transformations that mainly affect which target rows should be selected.

Definition 3.3 (Changed-Row Effect)**.** Given a DML statement q and a database state S, let $S' = [\![q]\!](S)$ be the post-state produced by executing q on S. The changed-row effect of q, denoted by $E_c(q, S)$, is the set of target-row tokens obtained by comparing S and S' and extracting the rows whose observable states differ before and after execution.

Example 3.2. Consider two tables in database state S:

$t1(c_1, c_2) =$

c_1	c_2
1	a
2	b

$t2(c_1, c_2, c_3) =$

c_1	c_2	c_3
5	x	1
6	y	2
7	z	-1

Consider the DML statement:

q : **INSERT INTO** $t1(c_1, c_2)$ **SELECT** c_1, c_2 **FROM** $t2$ **WHERE** $c_3 > 0$

Let $S' = [\![q]\!](S)$ be the post-state after execution. The newly inserted rows correspond to primary keys $\{5, 6\}$. Let h_5 and h_6 denote their normalized row summaries. We use a normalization function $row(\cdot)$ that maps a tuple to a canonical representation (e.g., a hash of its attribute values). Then the changed-row effect is:

$$E_c(q, S) = \{t1{:}5{:}ROW(h_5),\ t1{:}6{:}ROW(h_6)\}.$$

Intuitively, the effect cannot be derived purely from structure, and must be obtained by comparing the database states before and after execution.

The changed-row effect captures the *actual post-execution effect* of a DML statement. It is necessary when the mutation may influence value assignments, tuple multiplicity, insertion/deletion behavior, or other effects that cannot be determined reliably from the statement structure alone.

Effect Abstraction Selection. Accordingly, the abstract state effect $E(q, S)$ used by our approach is instantiated as either $E_m(q, S)$ or $E_c(q, S)$, depending on the comparison strategy. The distinction is essential: $E_m(q, S)$ characterizes which target rows are supposed to be affected according to the statement structure, whereas $E_c(q, S)$ characterizes which rows are actually changed in the database after execution.

More importantly, the choice between $E_m(q, S)$ and $E_c(q, S)$ is not determined merely by the DML type, but by the semantics targeted by the mutation and the robustness of the corresponding effect abstraction. In general, $E_c(q, S)$ is more expressive, because it reflects actual post-execution outcomes and can capture not only row selection but also value updates, insertions, deletions, and multiplicity-related effects. However, this stronger expressiveness may also introduce irrelevant execution-level variability. For example, in join-based `UPDATE` statements, one-to-many matches or unstable source-to-target mappings may lead to ambiguous value assignments, even when the intended target-row boundary is clear. In such cases, changed-row effects may become noisy or unreliable with respect to the mutation semantics under test. By contrast, $E_m(q, S)$ is less expressive but often more stable: it abstracts away concrete value changes and focuses only on the structurally intended target set. We next provide a concrete example to illustrate this trade-off and Section 4.6 then presents the detailed strategy for selecting the effect abstraction and for extracting state effects in practice.

Example 3.3. Consider the following tables in database state S:

$t_1(c_1, c_2) =$

c_1	c_2
1	0
2	0

$t_2(c_1, c_3) =$

c_1	c_3
1	100
1	200
2	300

Consider the statement

q : **UPDATE** t_1 **JOIN** t_2 **ON** $t_1.c_1 = t_2.c_1$ **SET** $t_1.c_2 = t_2.c_3$.

Structurally, the target rows in t_1 matched by q are unambiguous:

$$E_m(q, S) = \{t_1{:}1,\ t_1{:}2\}.$$

However, the changed-row effect depends on how the one-to-many join for key $c_1 = 1$ is resolved during execution. If the value 100 is chosen, one possible changed-row effect is

$$E_c(q, S) = \{t_1{:}1{:}ROW(h_{100}),\ t_1{:}2{:}ROW(h_{300})\},$$

whereas if the value 200 is chosen, another possible changed-row effect is

$$E_c(q, S) = \{t_1{:}1{:}ROW(h_{200}),\ t_1{:}2{:}ROW(h_{300})\}.$$

This example shows that changed-row effects may be more expressive but also less stable, since they may incorporate assignment-level ambiguity that is irrelevant when the mutation only targets row-selection semantics. Such ambiguity may arise from DBMS-specific source-row selection policies or from execution-dependent factors, such as different join evaluation orders or execution plans, which can lead the same target row to be assigned from different matching source rows. In contrast, matched-row effects remain stable because they only record the structurally intended target rows.

3.2 Approximate State Relation

To generalize approximation-based metamorphic testing to state-modifying statements, we extend the notion of approximation from query result sets to database state effects.

Definition 3.4 (Approximate State Relation)**.** Given a database state S, let q_1 and q_2 be two DML statements whose state effects are $E(q_1, S)$ and $E(q_2, S)$, respectively. We say that q_1 is the *state-level under-approximation* of q_2 over S, denoted by

$$q_1 \preceq_S q_2,$$

if and only if

$$E(q_1, S) \subseteq E(q_2, S).$$

Conversely, q_1 is the *state-level over-approximation* of q_2 over S, denoted by

$$q_1 \succeq_S q_2,$$

if and only if

$$E(q_1, S) \supseteq E(q_2, S).$$

Here, $E(q, S)$ denotes the state effect induced by executing statement q on state S, and $\subseteq$ and $\supseteq$ denote inclusion and containment relations between two effect sets.

Example 3.4. Consider a table $t(c_1, c_2)$ in database state S:

$t =$

c_1	c_2
1	10
2	20
3	30

Let the following two DML statements be defined:

$$q_1 : \textbf{UPDATE } t \textbf{ SET } c_2 = c_2 + 1 \textbf{ WHERE } c_1 > 1$$

$$q_2 : \textbf{UPDATE } t \textbf{ SET } c_2 = c_2 + 1 \textbf{ WHERE } c_1 \geq 1$$

If the state effect is instantiated as the matched-row effect, then

$$E(q_1, S) = \{t{:}2,\ t{:}3\}, \qquad E(q_2, S) = \{t{:}1,\ t{:}2,\ t{:}3\}.$$

Since

$$E(q_1, S) \subseteq E(q_2, S),$$

we have

$$q_1 \preceq_S q_2.$$

Intuitively, q_2 weakens the selection predicate and therefore affects a superset of the rows affected by q_1.

This definition lifts set-semantic approximation from query outputs to state transitions, enabling reasoning about correctness in terms of persistent database effects. To systematically construct DML statements with predictable approximation relations, we introduce approximate mutators.

Definition 3.5 (Approximate Mutator)**.** Let q be a DML statement and let n be a syntactic construct in q. An *approximate mutator* μ transforms q into a new statement $q' = \mu(q)$ by modifying n such that:

$$q \preceq_S q' \quad \text{or} \quad q \succeq_S q'.$$

The direction of the induced relation determines whether μ is an under- or over-approximate mutator.

Example 3.5. Consider the statement

$$q : \textbf{DELETE FROM } t \textbf{ WHERE } c_1 > 1.$$

A mutator may weaken the predicate to

$$q' : \textbf{DELETE FROM } t \textbf{ WHERE } c_1 \geq 1.$$

This transformation enlarges the set of affected rows, resulting in

$$q \preceq_S q'.$$

Approximate mutators provide a principled way to generate related statements whose semantic relationships are known *a priori*, which is essential for constructing effective metamorphic tests. However, a mutation typically affects only a local component of a statement. We therefore introduce approximation propagation to characterize how local changes determine global state-effect relations.

Definition 3.6 (Approximation Propagation)**.** Let q be a DML statement and let q' be obtained by applying an approximate mutator to a local construct $n \in q$. Approximation propagation describes how the local semantic deviation induced at n determines a global approximate state relation between q and q':

$$q \preceq_S q' \quad \text{or} \quad q \succeq_S q'.$$

Example 3.6. Consider a table $t(c_1, c_2)$ in database state S:

$t =$

c_1	c_2
1	5
2	10
3	15

Consider the DML statement

$$q : \textbf{DELETE FROM } t \textbf{ WHERE } \neg(c_2 > 10).$$

Since the predicate $\neg(c_2 > 10)$ is equivalent to $c_2 \leq 10$, the affected rows are

$$E(q, S) = \{t{:}1,\ t{:}2\}.$$

Now consider a local mutation that replaces the comparison operator $>$ with $\geq$ inside the negation:

$$q' : \textbf{DELETE FROM } t \textbf{ WHERE } \neg(c_2 \geq 10).$$

Although the local mutation $> \rightarrow \geq$ weakens the comparison expression itself, the surrounding negation reverses the approximation direction. Indeed, $\neg(c_2 \geq 10)$ is equivalent to $c_2 < 10$, and thus

$$E(q', S) = \{t{:}1\}.$$

Therefore,

$$E(q', S) \subseteq E(q, S),$$

which means that q' is a state-level under-approximation of q. This example illustrates that a local mutation may induce the opposite approximation direction at the statement level when it is propagated through a negation context.

Approximation propagation connects local syntactic transformations with global semantic relations, ensuring that approximation reasoning remains sound at the level of full DML statements.

3.3 Test Oracle

Based on the above definitions, ASRMT uses inclusion relations over state effects as the test oracle.

Definition 3.7 (State-Relation Oracle)**.** Given a seed statement q and its approximate variant q', a logical bug is detected if the observed state effects violate the expected inclusion relation:

- Over-approximation: $E(q, S) \not\subseteq E(q', S)$
- Under-approximation: $E(q', S) \not\subseteq E(q, S)$

This formulation enables approximation-based reasoning over persistent database behaviors, extending metamorphic testing from query correctness to state-transition correctness.

3.4 Discussion

ASRMT differs fundamentally from prior approximation-based metamorphic testing frameworks Pinolo [11], which are designed for read-only queries and define approximation relations over returned result sets. For DML statements, however, correctness is no longer determined by query outputs, but by how execution changes the underlying database state. This shift makes it necessary to redefine the semantic object of approximation as state effect, rather than tuples in a result set, and to reason about approximation in terms of state transitions. Moreover, because DML semantics involve both intended target-row selection and actual post-execution changes, ASRMT must support different effect abstractions and propagate local mutations to the statement level under state-based semantics, which substantially distinguishes it from prior read-only formulations.

4 Approach

Based on the ASRMT model introduced in Section 3, we design DMLScope, a practical testing framework for detecting logical bugs in DML statements. The key idea of DMLScope is to instantiate approximate state relations over executable DML statements, execute the original and mutated statements from the same initial database state, and check whether their observed state effects satisfy the expected inclusion relation. In the following, we present the overall workflow of DMLScope and then describe each core component in detail.

4.1 Overview

Figure 1 shows the overall workflow of DMLScope. The entire process follows a state-oriented generate–mutate–execute–verify paradigm. Starting from a round-specific database pre-state, DMLScope first generates an executable seed DML statement, then synthesizes its approximate variants according to the approximation relations defined in Section 3. Next, all statements are executed under differential state execution, where each statement is executed independently from the same pre-state and the database state is restored after each execution, so that their resulting state effects are directly comparable. Finally, DMLScope extracts state-effect tokens and checks whether the observed effects satisfy the expected approximate state relation. Any violation of the expected relation indicates a potential logical bug.

In the following subsections, we describe these components in detail, including database initialization (Section 4.2), original DML generation (Section 4.3), approximate DML synthesis (Section 4.4), differential state execution (Section 4.5), state-effect extraction (Section 4.6), and approximate state relation checking (Section 4.7).

4.2 Database Initialization

Before generating and testing DML statements, ASRMT first constructs an independent round environment by initializing a database with a valid schema and initial data. This step is not a primary contribution of our work, but is necessary to ensure the correctness and executability of subsequent testing stages. Specifically, we first generate a relational schema consisting of multiple tables with primary key and foreign key constraints, and map logical column types to DBMS-specific physical types to ensure compatibility across different systems. We then populate each table with a small number of rows while satisfying integrity constraints such as key uniqueness and referential dependencies. The insertion order follows dependency relationships so that referenced tables are initialized before dependent ones. In addition, we maintain auxiliary metadata, such as the set of existing primary keys, to guide the generation of valid DML statements and avoid referencing non-existent tuples. This design ensures that all generated statements are executable on the constructed database state and provides a stable pre-state for subsequent differential state execution.

4.3 Original DML Generation

To systematically explore diverse state-modifying behaviors, we first generate syntactically valid and executable DML statements as testing seeds. This process consists of two stages: *Grammar-guided structure generation*, which constructs abstract syntax trees (ASTs) according to a predefined grammar, and *Schema-based instantiation*, which binds schema elements and concrete values to produce executable statements. Compared with read-only query generation, original DML generation must additionally guarantee the validity of state-modifying semantics. If a DML statement that violates integrity constraints, such as `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, or `FOREIGN KEY`, may abort entirely or cause rollback, making its state effect undefined and therefore unusable for subsequent approximation checking. Algorithm 1 summarizes the overall procedure.

Grammar-Guided Structure Generation. Initially, the algorithm performs *grammar-guided structure generation* (Lines 1–4), which determines the abstract syntactic shape of a seed DML statement. It first selects a statement kind k (Line 2), then chooses a compatible target table T (Line 3), and finally expands the corresponding grammar rules to build an abstract statement skeleton A (Line 4). We use the compact DML grammar in Figure 2 to guide this process. Beyond basic DML forms, the grammar also supports joins, nested subqueries, common table expressions, and set operations, enabling the generation of structurally rich statements. The generator recursively samples productions to construct an AST, while bounding recursion depth and restricting highly combinatorial constructs to keep generation tractable. At this stage, only the structural skeleton is determined; concrete schema bindings and values are instantiated later.

Schema-based Instantiation. Next, the algorithm performs *schema-based instantiation*, which concretizes the abstract skeleton into an executable DML statement under the current schema and database pre-state. It begins by generating candidate target columns from the skeleton and the target-table schema (Line 6), and then filters out invalid columns that are not writable, not visible in the current scope, or incompatible with the chosen statement kind (Line 7). Based on the remaining column candidates, the algorithm generates source candidates that may provide values, bindings, or filtering contexts for the statement (Line 8), and removes unstable or incompatible sources by enforcing source-side semantic constraints (Line 9). It then materializes a consistent column–source plan (π_c, π_s) (Line 10), which serves as the blueprint for the subsequent instantiation process. After the plan is fixed, the algorithm binds concrete schema elements to the skeleton (Line 11), and then

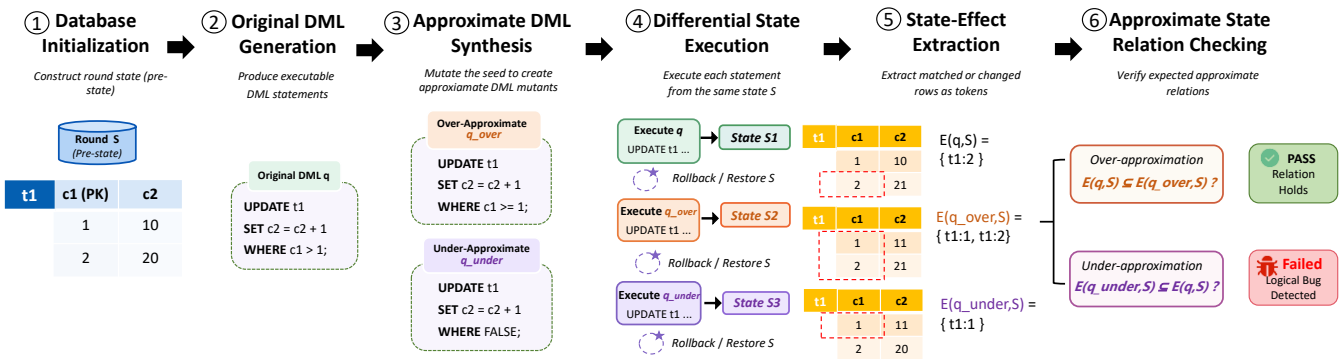

Figure 1: Overview of DMLScope.

incrementally instantiates key positions, non-key values, and predicates (Lines 12–14). This order is important. Key positions are instantiated first because they determine tuple identity and may immediately conflict with the current pre-state. Once key values are fixed, the algorithm instantiates non-key payloads under source, type, and nullability constraints, and finally fills predicate slots with scope-correct and semantically meaningful filtering conditions. In this way, the generator does not first produce an arbitrary statement and then attempt to repair it afterwards; instead, it progressively constructs the statement while preserving semantic consistency at each stage. The algorithm then performs DML-specific semantic checks according to the statement kind (Lines 15–22). For INSERT and REPLACE, it checks key consistency and source–payload alignment (Lines 15–17), ensuring that inserted tuples neither violate uniqueness constraints nor introduce incompatible source-to-target mappings. For UPDATE, it checks assignment validity (Lines 18–19), so that writable target columns, source bindings, and right-hand-side expressions remain jointly meaningful. For DELETE, it checks that the predicate is non-degenerate (Lines 20–21), avoiding statements that collapse into trivial or uncontrolled deletions. These statement-specific checks are essential because, unlike read-only query generation, DML generation must additionally preserve the validity of state-modifying semantics. Finally, the algorithm performs structural checking, serialization, and execution validation (Lines 23–28). It first verifies that the instantiated AST remains structurally complete and internally consistent (Line 23), and then serializes it into a concrete SQL statement q (Line 24). The statement is accepted only if it can be executed successfully on the current pre-state S (Lines 25–26); otherwise, the generation process restarts (Lines 27–28).

Figure 3 presents a simplified example of original DML generation for an UPDATE statement. In this example, the generator produces an abstract skeleton with unresolved table, column, and predicate slots. During the schema-based instantiation stage, the generator first selects t_1 as the target table and t_2 as the source table, because the two tables can be naturally connected through the foreign-key relationship on column c_1. It then fills the assignment slots with $t_1.c_2$ and $t_2.c_2$, since the former is writable under UPDATE semantics and the latter provides a type-compatible source value. Next, it instantiates the predicate slot as $t_1.c_1 = t_2.c_1$, so that the abstract join becomes a semantically meaningful row-to-row

Algorithm 1 Original DML Generation

Require: Schema $\mathcal{S}$, current database pre-state S
Ensure: Original DML statement q
1: // **Step 1: Grammar-guided structure generation**
2: Choose statement kind $k \in \mathcal{K}$, where $\mathcal{K} = \{\texttt{UPDATE}, \texttt{DELETE}, \texttt{INSERT}, \texttt{REPLACE}\}$.
3: Select target table $T \leftarrow \textsc{SelectTarget}(\mathcal{S}, k)$.
4: Build abstract skeleton $A \leftarrow \textsc{ExpandGrammar}(k, T)$.
5: // **Step 2: Schema-based instantiation**
6: Generate target-column candidates $C \leftarrow \textsc{GenColCands}(A, \mathcal{S}[T])$.
7: Filter invalid columns $C \leftarrow \textsc{FilterCols}(C, k, \mathcal{S}[T])$.
8: Generate source candidates $\mathcal{R} \leftarrow \textsc{GenSrcCands}(A, \mathcal{S}, S, C)$.
9: Filter unstable sources $\mathcal{R} \leftarrow \textsc{FilterSrcs}(\mathcal{R}, C)$.
10: Materialize a consistent plan $(\pi_c, \pi_s) \leftarrow \textsc{BindPlan}(C, \mathcal{R})$.
11: Bind schema elements $A \leftarrow \textsc{BindSchema}(A, \pi_c, \pi_s)$.
12: Instantiate key positions $A \leftarrow \textsc{InstKey}(A, S, \pi_c, \pi_s)$.
13: Instantiate non-key values $A \leftarrow \textsc{InstVal}(A, S, \pi_c, \pi_s)$.
14: Instantiate predicates $A \leftarrow \textsc{InstPred}(A, S, \pi_s)$.
15: **if** $k \in \{\texttt{INSERT}, \texttt{REPLACE}\}$ **then**
16: $\quad A \leftarrow \textsc{CheckKeyConsistency}(A, S)$
17: $\quad A \leftarrow \textsc{CheckSrcPayloadAlignment}(A, \pi_s)$
18: **else if** $k = \texttt{UPDATE}$ **then**
19: $\quad A \leftarrow \textsc{CheckAssignmentValidity}(A, \mathcal{S}[T])$
20: **else if** $k = \texttt{DELETE}$ **then**
21: $\quad A \leftarrow \textsc{CheckNonDegeneratePred}(A, \pi_s)$
22: **end if**
23: $A \leftarrow \textsc{CheckStructure}(A, k)$
24: $q \leftarrow \textsc{Serialize}(A)$.
25: **if** $\textsc{ExecValid}(q, S)$ **then**
26: $\quad$ **return** q
27: **else**
28: $\quad$ restart generation
29: **end if**

correspondence between the two tables. As a result, the final original DML statement is structurally complete, type-consistent, and executable on the current database state.

4.4 Approximate DML Synthesis

Given an executable original DML statement, DMLScope next synthesizes a set of approximate variants whose state effects are expected to follow a known directional relation with respect to the seed statement, i.e., either over-approximation or under-approximation. This process is mutation-based: instead of generating new statements from scratch, the algorithm systematically applies approximate mutators to selected components of the original DML.

Table 1 summarizes some representative approximate mutators used in our synthesis phase. Our design follows the general spirit of

DML	D	::=	$U \mid Del \mid Ins \mid Rep$
WithClause	W	::=	WITH id AS (SQ) (, id AS (SQ))*
JoinType	J	::=	JOIN | INNER JOIN | LEFT JOIN | RIGHT JOIN
SetOp	SO	::=	UNION | UNION ALL | EXCEPT | INTERSECT
Update	U	::=	[W] UPDATE id [PARTITION(id (, id)*)] [J SRC [AS id] ON P] SET $ASSIGNS$ [FROM SRC [AS id]] WHERE P
Delete	Del	::=	[W] DELETE FROM id [PARTITION(id (, id)*)] WHERE P
		|	[W] DELETE FROM id J SRC ON P WHERE P
		|	[W] DELETE FROM id USING SRC WHERE P
Insert	Ins	::=	INSERT INTO id ($COLS$) IB
		|	INSERT INTO id ($COLS$) IB CF
Replace	Rep	::=	(REPLACE | INSERT OR REPLACE) INTO id ($COLS$) IB
InsertBody	IB	::=	VALUES ($EXPRS$) | IQ
InsertQuery	IQ	::=	$SQ \mid SS$
SelectQuery	SQ	::=	[W] SELECT [DISTINCT] $EXPRS$ FROM SRC [WHERE P] [GROUP BY $EXPRS$ [HAVING P]] [ORDER BY $EXPRS$] [LIMIT int]
SelectSet	SS	::=	$SQ \mid SQ\ SO\ SQ$
Source	SRC	::=	id [AS id]
		|	id [AS id] J (id | (SQ) | (SS)) [AS id] ON P
		|	(SQ) AS id | (SS) AS id
Predicate	P	::=	$E\ \theta\ E$ | NOT P | P AND P | P OR PEXISTS(SQ)
		|	E IS [NOT] NULL | E [NOT] IN(SQ | $EXPRS$)
Expr	E	::=	$id \mid const \mid f(E) \mid (\ SS\ )$
Assignments	$ASSIGNS$	::=	$id = E$ (, $id = E$)*
Columns	$COLS$	::=	id (, id)*
ExprList	$EXPRS$	::=	E (, E)*
Conflict	CF	::=	ON DUPLICATE KEY UPDATE $ASSIGNS$
		|	ON CONFLICT (id) DO NOTHING
		|	ON CONFLICT (id) DO UPDATE SET $ASSIGNS$
CompOp	θ	::=	= | <> | < | > | <= | >= | LIKE | BETWEEN

Figure 2: Compact DML Syntax Supported by DMLScope. For brevity, low-level lexical elements, including identifiers, function symbols, and constants, are abstracted as *id*, *f*, and *const*.

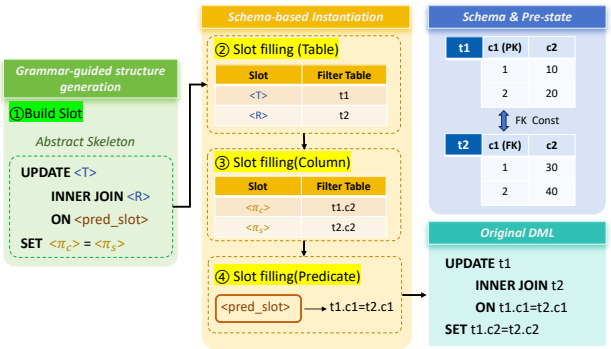

Figure 3: Illustrative Example of Original DML Generation.

Pinolo [11], which organizes mutators around semantic approximation, while extending it to better support DML-specific constructs. In particular, beyond relation-, predicate-, and comparison-oriented mutations, we additionally introduce *Join / Structure* mutators to handle DML-relevant constructs such as join type and HAVING-related query structure. Overall, compared with Pinolo, our framework introduces XX additional mutators; more complete mutation rules are available in our GitHub repository.

Algorithm 2 summarizes the overall approximate DML synthesis procedure. Given an executable original DML statement q, the algorithm first parses it into an AST and identifies a set of semantically meaningful local constructs that admit approximate mutation (Lines 2–3), including predicates, comparison expressions, join conditions, and query-structure constructs. It then iterates over these constructs and enumerates applicable approximate mutators for each of them (Lines 6–8). For every selected mutator, the algorithm applies the mutation at the AST level and serializes the resulting structure into a concrete DML statement q' (Lines 9–10). Next, it determines the approximation direction in two stages: it first infers the local direction induced at the mutation site by the selected mutator (Line 11), and then propagates this direction through the surrounding syntactic context to obtain the global approximation direction of the mutated statement (Line 12). In particular, when the mutation site appears in direction-reversing contexts, such as NOT, NOT EXISTS, or the right branch of EXCEPT, the approximation direction is reversed, following the same propagation principle as in Pinolo [11].

Algorithm 2 Approximate DML Synthesis

Require: Original DML statement q
Ensure: A set of approximate variants $\mathcal{R}$
1: // **Step 1: Parse and collect mutation candidates**
2: Parse q into AST A
3: $\Gamma \leftarrow$ CollectMutableNodes(A)
4: // **Step 2: Apply approximate mutators**
5: $\mathcal{R} \leftarrow \emptyset$
6: **for all** node $n \in \Gamma$ **do**
7: $\quad\Omega \leftarrow$ ApplicableMutators(n)
8: $\quad$**for all** mutator $\mu \in \Omega$ **do**
9: $\quad\quad A' \leftarrow$ ApplyMutation(A, n, μ)
10: $\quad\quad q' \leftarrow$ Serialize(A')
11: $\quad\quad m_\ell \leftarrow$ InferLocalDirection(μ)
12: $\quad\quad m \leftarrow$ PropagateDirection(m_ℓ, n, A)
13: $\quad\quad \mathcal{R} \leftarrow \mathcal{R} \cup \{(q', m)\}$
14: $\quad$**end for**
15: **end for**
16: **return** $\mathcal{R}$

Table 1: Some representative approximate mutators for DML synthesis. Transforming the construct C_1 into C_2 achieves the under-approximation, while transforming C_2 into C_1 achieves the over-approximation.

Type	C_1	C_2
Relation	SELECT ... A UNION ALL B A UNION B A	SELECT DISTINCT ... A UNION B A INTERSECT B A EXCEPT B
Predicate	P TRUE $P \wedge Q$ JOIN ... ON P	FALSE P $P \vee Q$ JOIN ... ON TRUE
Comparison	$a > b$ $a < b$ x BETWEEN l AND r $e \odot$ ANY(r)	$a \geq b$ $a \leq b$ $x \geq l$ $e \odot$ ALL(r)
Join / Structure	INNER JOIN HAVING P GROUP BY ... HAVING P	LEFT JOIN HAVING TRUE GROUP BY ...

Algorithm 3 Differential State Execution

Require: Database pre-state S, original DML statement q, approximate variants $\mathcal{R}$
Ensure: A set of post-states $\mathcal{X} = \{(q_i, S'_i)\}$
1: $\mathcal{Q} \leftarrow \{q\} \cup \{q' \mid (q', m) \in \mathcal{R}\}$
2: $\mathcal{X} \leftarrow \emptyset$
3: **for all** statement $q_i \in \mathcal{Q}$ **do**
4: $\quad$ RestoreState(S)
5: $\quad S'_i \leftarrow [\![q_i]\!](S)$
6: $\quad \mathcal{X} \leftarrow \mathcal{X} \cup \{(q_i, S'_i)\}$
7: **end for**
8: **return** $\mathcal{X}$

4.5 Differential State Execution

To make state effects directly comparable across related statements, DMLScope executes the original DML statement and all its approximate variants under differential state execution. The key requirement is that every statement must be evaluated from the same database pre-state S, rather than from a state already modified by another execution. This requirement is essential for ASRMT because the expected approximate state relation is defined between $E(q, S)$ and $E(q', S)$, i.e., the state effects of two statements over the same initial state. If one statement is executed after another without restoring the database, the resulting effect would instead be measured over a different state, making the subsequent comparison semantically invalid.

Algorithm 3 summarizes the overall differential state execution procedure. Given a database pre-state S, an original DML statement q, and its synthesized approximate variants $\mathcal{R}$, the algorithm first materializes a statement set $\mathcal{Q}$ consisting of the original statement and all mutated statements (Line 1). It then iterates over each statement $q_i \in \mathcal{Q}$ and executes it independently from the same pre-state (Lines 3–6). Concretely, before each execution, the algorithm restores the database to S (Line 4), so that every statement observes an identical input state. It then executes the current statement to obtain its post-state S'_i (Line 5) and records this post-state as the independent execution result of q_i (Line 6). This design isolates each execution from the side effects of all others, ensuring that the post-states later used for state-effect extraction are produced from a common starting point rather than from states modified by previous executions. In practice, we implement the restoration step through dual isolation. First, each statement is executed in a separate subprocess, which isolates executions at the runtime level and prevents side effects from propagating through shared process state. Second, within each subprocess, the statement is executed inside an isolated transaction and the transaction is rolled back after execution, thereby restoring the database to the round-level pre-state S. Together, these two mechanisms ensure that every original or approximate DML statement is evaluated from the same clean execution environment. The resulting post-states are then passed to the next stage for state-effect extraction.

4.6 State-Effect Extraction

After differential state execution, DMLScope extracts state-effect tokens from each executed statement, which serve as the canonical representation for subsequent relation checking. Algorithm 4 summarizes the overall state-effect extraction procedure.

Given the execution results $\mathcal{X} = \{(q_i, S'_i)\}$ produced in Section 4.5, the algorithm processes each statement independently. For

Algorithm 4 State-Effect Extraction

Require: Execution results $\mathcal{X} = \{(q_i, S'_i)\}$, pre-state S, mutation context (μ, n)
Ensure: State-effect token sets $\{\mathcal{T}_i\}$
1: **for all** $(q_i, S'_i) \in \mathcal{X}$ **do**
2: $\quad dml \leftarrow$ ParseDml(q_i)
3: $\quad basis \leftarrow$ SelectBasis(μ, n, dml, S)
4: $\quad pk_only \leftarrow$ NeedPKOnly$(\mu, n, basis)$
5: $\quad strategy \leftarrow (basis, pk_only)$
6: $\quad$ **if** $basis$ = `matched_rows` **then**
7: $\quad\quad rows \leftarrow$ Execute$(dml.match_sql)$
8: $\quad\quad \mathcal{T}_i \leftarrow$ MatchToTokens$(rows, pk_only)$
9: $\quad$ **else**
10: $\quad\quad before \leftarrow$ Snapshot(S)
11: $\quad\quad after \leftarrow$ Snapshot(S'_i)
12: $\quad\quad \mathcal{T}_i \leftarrow$ DiffToTokens$(before, after, pk_only)$
13: $\quad$ **end if**
14: **end for**
15: **return** $\{\mathcal{T}_i\}$

each q_i, it first parses the statement into a structured representation (Line 2), which includes its statement kind, target table, primary key, and (if available) a derived matching query (`match_sql`). Based on the mutation operator and its syntactic context, the algorithm then determines an extraction strategy (Lines 3–5), which specifies both the extraction basis (`matched_rows` or `changed_rows`) and whether only primary keys should be considered. ✎ [lin: add more detailes about effect selection] The extraction follows two alternative paths depending on the selected strategy. If the `matched_rows` strategy is applicable, the algorithm derives the target-row set directly from the statement structure (Lines 6–7). Specifically, it executes the derived `match_sql` query to obtain the primary keys of all structurally matched rows, and converts them into tokens of the form `table:pk`. This path avoids dependence on execution results and is therefore more stable when the mutation primarily affects row-selection semantics. Otherwise, the algorithm adopts the `changed_rows` strategy (Lines 8–11), which extracts effects from actual state differences. It first reconstructs the pre-state snapshot S and compares it with the post-state S'_i obtained from differential execution. Rows whose observable representations differ between the two states are identified as changed rows. Each such row is normalized into a token of the form `table:pk:ROW(hash)`, capturing both row identity and value changes. This strategy is necessary when the mutation may influence value assignments, tuple multiplicity, or insertion/deletion behavior. Finally, all extracted tokens are normalized into a set representation (Line 12), referred to as the state-effect token set, which serves as a concrete representation of the state effect $E(q_i, S)$. These token sets abstract away DBMS-specific execution details while preserving the essential semantics required for approximation checking, and are used in the next stage to verify expected over- or under-approximation relations.

Figure 4 illustrates two representative scenarios for state-effect extraction. Consider the `UPDATE` statement shown in Figure 4(a), where the mutation only affects the join predicate (e.g., $t1.c1 \leq t2.c1$). Although different executions may produce different assigned values due to multiple matching rows in $t2$, the set of affected target rows in $t1$ remains invariant. In this case, extracting state effects from execution results would introduce unnecessary instability. Therefore, the algorithm selects the `matched_rows` strategy and derives a matching query (`match_sql`) from the statement structure. By executing this query, it obtains the set of structurally

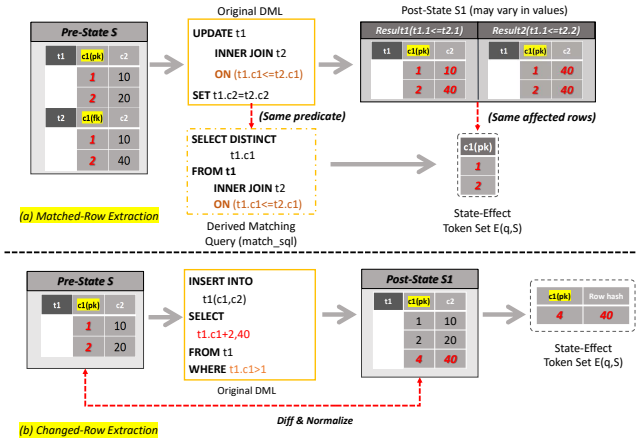

Figure 4: Illustrative Example of State-Effect Extraction.

matched primary keys $\{1, 2\}$, which directly forms the state effect $E(q, S)$. In contrast, Figure 4(b) shows an `INSERT` statement, where the mutation influences both tuple generation and value assignments. Here, the affected rows cannot be determined purely from the query structure, as the inserted tuples depend on the execution outcome. Thus, the algorithm selects the `changed_rows` strategy. It compares the pre-state S and the post-state $S1$ to identify newly inserted rows (e.g., $(4, 40)$), and normalizes them into tokens of the form `table:pk:ROW(hash)`. These tokens capture both row identity and value changes, yielding the state effect $E(q, S)$.

4.7 Approximate State Relation Checking

After state-effect extraction, DMLScope verifies whether the extracted token sets satisfy the expected approximate state relation. For an over-approximate mutant, the token set of the original statement should be contained in that of the mutant, whereas for an under-approximate mutant, the token set of the mutant should be contained in that of the original statement. Any violation of the expected relation indicates a potential logical bug.

4.8 Other Implementation Details

We implemented DMLScope as a prototype DBMS testing system in Python, consisting of 13,455 lines of code. To support mutation at the structural level, DMLScope leverages SQLGlot [20] to parse original DML statements into ASTs, enabling precise application of approximate mutators while preserving syntactic and semantic correctness. During testing, DMLScope executes generated statements on target DBMSs and collects execution results for further analysis. To improve the usability of detected bugs, we adopt SQLess [18] to simplify failing cases and isolate their root causes, and further apply a deduplication strategy following Pinolo [11] to eliminate redundant bug reports. DMLScope is primarily designed for MySQL-compatible systems, including TiDB and OceanBase, but its modular architecture allows for straightforward adaptation to other DBMS dialects by customizing syntax- and type-related components. In practice, such adaptation requires only lightweight modifications, and can be further automated using tools such as QTRAN [19], which facilitates cross-dialect deployment of metamorphic testing techniques.

5 Evaluation

Our evaluation aims to answer the following questions to demonstrate the effectiveness of DMLScope:

- ***Q1*** How many unique and new bugs in real-world DBMSs can DMLScope detect? (Section 5.2)
- ***Q2*** How diverse are the logical bugs found by DMLScope? (Section 5.3)
- ***Q3*** Can DMLScope find logical bugs that are missed by existing approaches? (Section 5.4)

5.1 Experimental Setup

Environment. We conduct the experiments on one server with 104-cores Intel(R) Xeon(R) Gold 6230R CPU @2.10GHz and 500 GB memory. The server operates under the Ubuntu 20.04 OS, with the 5.4.0-135-generic version of the Linux kernel.

5.2 Bug Detection

5.3 Comprehensive Bug Analysis

5.4 Comparative Study

6 Related Work

Logical Bug Detection in DBMSs. A variety of approaches have been proposed to detect logical bugs in DBMSs [3, 7, 11, 14, 15, 26–28, 30–32]. For example, NoREC [26] transforms an optimizable query into a non-optimizable form and detects semantic inconsistencies by comparing their outputs. TLP [27] divides query predicates into multiple subqueries and verifies that the union of their results is equivalent to the original query. PQS [28] constructs queries expected to retrieve a specific pivot row, while DQE [30] detects logical bugs by comparing whether different SQL queries with the same predicate access the same rows in the database. Pinolo [11] generalizes this paradigm through set-level approximation, where predicates are relaxed or restricted to generate over- and under-approximate queries, and correctness is judged by inclusion or containment relations between result sets. TQS [32] decomposes wide tables into smaller ones and uses the base table as ground truth for correctness validation. EET [15] applies expression-preserving transformations, and Radar [31] compares query results between databases with and without metadata to identify semantic flaws. EDC [7] detects logical bugs by substituting expressions with precomputed equivalent data and checking for inconsistent results, while CODDTest [35] leverages constant folding and propagation to generate equivalent queries.

As discussed in Section 1 and Section 2, these approaches mainly focus on validating query correctness through returned results, such as equivalence, containment, or cross-query consistency. In contrast, DMLScope focuses on the effects of DML execution on database states. Rather than reasoning about returned result sets, it examines whether state-modifying operations produce the expected effects. From this perspective, DMLScope is largely orthogonal to existing query-centric testing techniques, as it addresses a complementary aspect of DBMS correctness.

DBMS Test-Case Generation. A variety of approaches have been proposed for generating diverse test cases for DBMSs, with the aim of improving coverage and revealing potential bugs. These

techniques [1, 10, 13, 19, 36] typically focus on generating valid and varied SQL queries, but do not specifically target logical bugs. For example, SQLsmith [1] is a grammar-based DBMS fuzzer that embeds SQL grammar rules to generate complex SQL queries. It uses a random walk approach to explore the SQL syntax and generate a wide range of queries. SQUIRREL [36] is a mutation-based DBMS fuzzer which introduces an intermediate representation for SQL queries and models dependencies between SQL statements, enabling the generation of queries that contain multiple SQL operations. Griffin [10] uses a grammar-free mutation approach, where it mutates SQL queries based on DBMS state information encapsulated in a metadata graph. QTRAN [19] is a LLM-based approach that can automatically translate test cases from other DBMSs. DynSQL [13] takes a dynamic approach by interacting with the DBMS to capture the latest state information, allowing for the incremental generation of valid and complex queries. These techniques aim to prevent semantic errors and improve the diversity of generated queries. In addition to general query generation, some approaches have been specifically designed to aid in bug detection. SQLRight [17] leverages code coverage feedback to enhance test-case generation. QPG [2] takes a different approach by recording the query plans covered during DBMS testing and prioritizing the mutation of queries that trigger new query plans.

Different from these works, DMLScope does not primarily target general-purpose SQL generation or coverage improvement. Instead, it focuses on generating semantically valid DML seeds and approximate DML variants for testing state-modifying semantics. Its main goal is not merely to increase query diversity, but to expose subtle logical bugs in DML execution by checking whether state effects satisfy expected semantic relations. From this perspective, DMLScope is complementary to existing DBMS test-case generation techniques: prior generators can serve as upstream sources of diverse and valid SQL inputs, while DMLScope contributes a DML-specific testing methodology and oracle for detecting state-transition bugs.

7 Conclusion

References

[1] Anse1. 2020.. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed Dec 2025.

[2] Jinsheng Ba and Manuel Rigger. 2023. Testing database engines via query plan guidance. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2060–2071.

[3] Jinsheng Ba and Manuel Rigger. 2024. Keep it simple: Testing databases via differential query plans. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[4] Hardik Bati, Leo Giakoumakis, Steve Herbert, and Aleksandras Surna. 2007. A genetic approach for random testing of database systems. In *Proceedings of the 33rd international conference on Very large data bases*. 1243–1251.

[5] Donald D Chamberlin and Raymond F Boyce. 1974. SEQUEL: A structured English query language. In *Proceedings of the 1974 ACM SIGFIDET (now SIGMOD) workshop on Data description, access and control*. 249–264.

[6] Alibaba Cloud. 2023. PolarDB Product Overview. https://help.aliyun.com/zh/polardb/product-overview/. Accessed Dec 2025.

[7] Wenqian Deng, Jie Liang, Zhiyong Wu, Jingzhou Fu, and Yu Jiang. 2025. Detecting Logic Bugs in DBMSs via Equivalent Data Construction. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[8] Daniela Florescu, Alon Levy, and Alberto Mendelzon. 1998. Database techniques for the World-Wide Web: A survey. *ACM Sigmod Record* 27, 3 (1998), 59–74.

[9] MariaDB Foundation. 2023. MariaDB. https://mariadb.org/ Accessed Dec 2025.

[10] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang, and Yu Jiang. 2022. Griffin: Grammar-free dbms fuzzing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE 22)*. 1–12.

[11] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting logical bugs in database management systems with approximate query synthesis. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 345–358.

[12] William E Howden. 2006. Theoretical and empirical studies of program testing. *IEEE Transactions on Software Engineering* 4 (2006), 293–298.

[13] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. {DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4949–4965.

[14] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting transactional bugs in database engines via {graph-based} oracle construction. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 397–417.

[15] Zu-Ming Jiang and Zhendong Su. 2024. Detecting logic bugs in database engines via equivalent expression transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 821–835.

[16] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. 2023. Testing graph database engines via query partitioning. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 23)*. 140–149.

[17] Yu Liang, Song Liu, and Hong Hu. 2022. Detecting logical bugs of {DBMS} with coverage-based guidance. In *31st USENIX Security Symposium (USENIX Security 22)*. 4309–4326.

[18] Li Lin, Zongyin Hao, Chengpeng Wang, Zhuangda Wang, Rongxin Wu, and Gang Fan. 2024. SQLess: Dialect-Agnostic SQL Query Simplification. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 24)*. 743–754.

[19] Li Lin, Qinglin Zhu, Hongqiao Chen, Zhuangda Wang, Rongxin Wu, and Xiaoheng Xie. 2025. QTRAN: Extending Metamorphic-Oracle Based Logical Bug Detection Techniques for Multiple-DBMS Dialect Support. In *Proceedings of the 34nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 25)*. 731–752.

[20] Toby Mao. 2024. SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine. https://github.com/tobymao/sqlglot Accessed Dec 2025.

[21] MySQL. 2023. MySQL. https://www.mysql.com/ Accessed Dec 2025.

[22] OceanBase. 2023. OceanBase Open Source. https://www.oceanbase.com/product/opensource Accessed Dec 2025.

[23] Percona. 2023. Percona - Open Source Database Software and Services. https://www.percona.com/ Accessed Dec 2025.

[24] PingCap. 2022. go-randgen. https://github.com/pingcap/go-randgen Accessed Dec 2025.

[25] PingCAP. 2023. TiDB. https://www.pingcap.com/tidb/ Accessed Dec 2025.

[26] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152.

[27] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.

[28] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 667–682.

[29] Donald R Slutz. 1998. Massive stochastic testing of SQL. In *VLDB*, Vol. 98. 618–622.

[30] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing database systems via differential query execution. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2072–2084.

[31] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1884–1897.

[32] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting logic bugs of join optimizations in dbms. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.

[33] Rick F. Van Der Lans. 1989. *The SQL Standard: A Complete Guide Reference*. Prentice Hall International (UK) Ltd.

[34] Wikipedia. 2026. Databases. https://en.wikipedia.org/wiki/Database. Accessed: 2026-01-23.

[35] Chi Zhang and Manuel Rigger. 2025. Constant Optimization Driven Database System Testing. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–24.

[36] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 955–970.

[37] Xuanhe Zhou, Chengliang Chai, Guoliang Li, and Ji Sun. 2020. Database meets artificial intelligence: A survey. *IEEE Transactions on Knowledge and Data Engineering* 34, 3 (2020), 1096–1116.